

CS & IT ENGINEERING

Programming in C

Arrays and Pointers

Lec- 05



By- Pankaj Sharma sir



TOPICS TO BE
COVERED



Arrays and Pointers

Add_x is 2000 int $\Rightarrow$ 4 bytes

Gate - 2015

2 min

2-dimension
unsigned int $x[4][3] = \{ \{1, 2, 3\}, \{4, 5, 6\}, \{7, 8, 9\}, \{10, 11, 12\} \};$

printf("/u /u /u", $x+3$, $*(x+3)$, $*(x+2)+3$);

$*(a+i) \Rightarrow a[i]$

A) 2036, 2036, 2036

B) 2012, 4, 2024

C) 2036, 10, 10

D) 2012, 4, 6

$x+3$
Add + 3
Address

$x[3]$
1-dim

Address

$x[2] + 3$
1-dim

Add

$x[3]$

$\Rightarrow x[3][0]$

Add x is 2000 $\text{int} \Rightarrow 4$ bytes

Gate - 2015

2 min

2-dimension
unsigned int $x[4][3] = \{ \{1, 2, 3\}, \{4, 5, 6\}, \{7, 8, 9\}, \{10, 11, 12\} \};$

$\text{printf}("/u /u /u", \quad \underline{x+3}, \quad \underline{*(x+3)}, \quad \underline{*(x+2)+3});$

$*(a+i) \Rightarrow a[i]$

A) 2036, 2036, 2036

B) 2012, 4, 2024

C) 2036, 10, 10

D) 2012, 4, 6

$\underline{x+3}$

$\underline{\text{Add} + 3}$

Address

$x[3]$

1-dim

Address

$\Rightarrow x[3][0]$

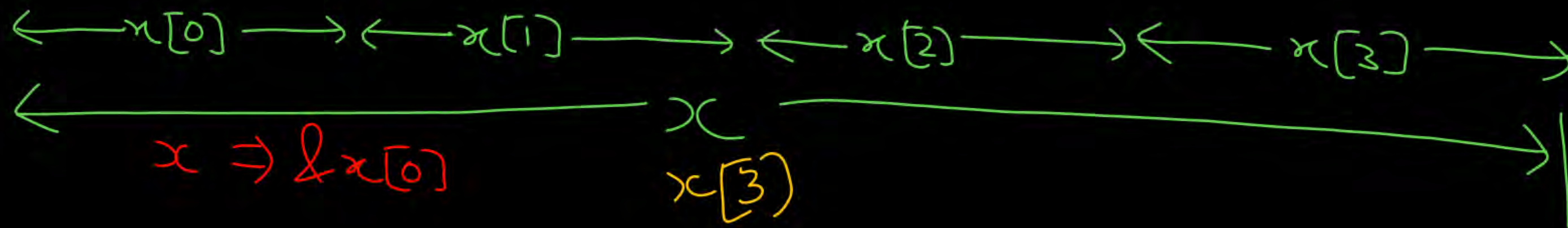
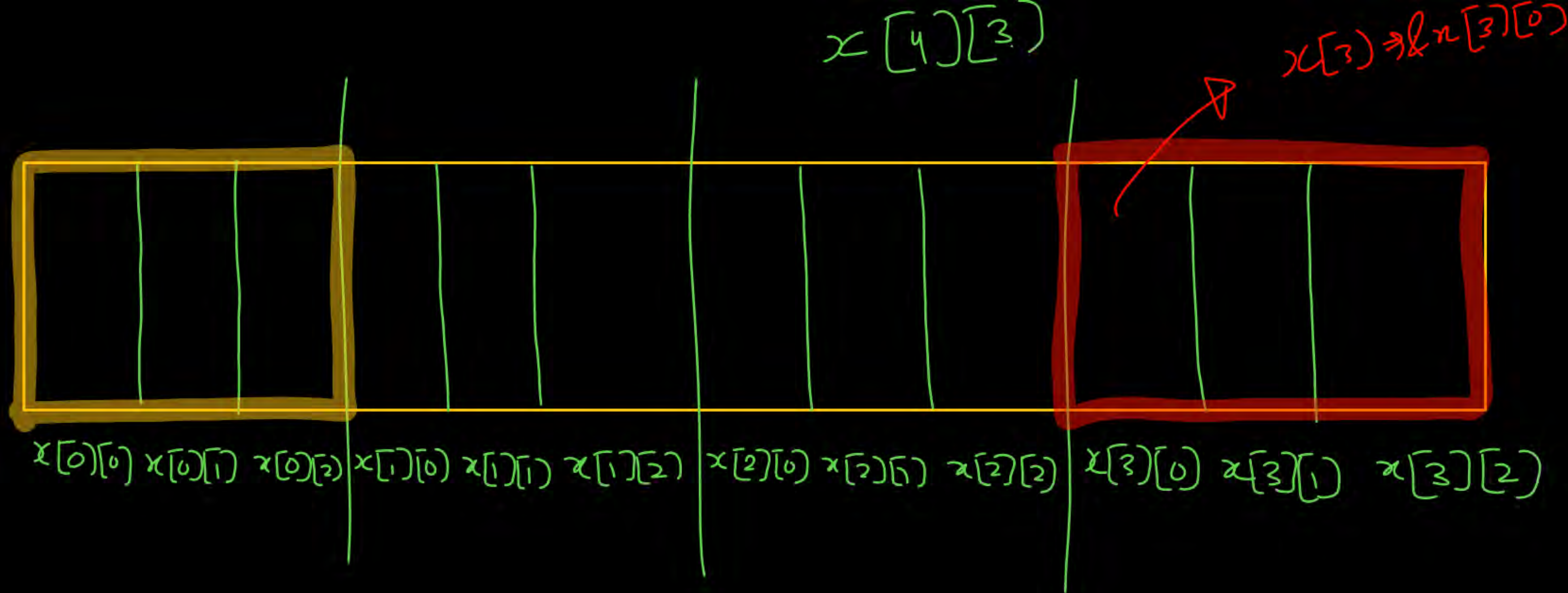
$x[2] + 3$

1-dim

Add

$x[3]$

$\Rightarrow x[3][0]$



Q

b.A
1000
int $\Rightarrow$ 4 byte

int a[4] = {10, 20, 30, 40, 50};

printf("/u", a); $\rightarrow$ &a[0]

printf("/u", a[0]); $\rightarrow$ 10

printf("/u", a+1); $\rightarrow$ $\underbrace{\&a[0]}_{1000} + 1 \times 4$

printf("/u", &a+1);

$\rightarrow$ $\&a + 1 \times 16$
 $= 1016$

A) 10 10 20 20

B) 1000 1000 20 20

C) 1000 10 20 1016

☒ D) 1000 10 1004 1016

E) None

b.A - 1000
int - 4 bytes

Q

int a[2][3] = {0, 2, 3, 4, 5, 6};

printf("/u /u /u", a, *a, **a);

printf("/u /u /u ", a+1, *a+1, **a+1);

A) 1000 1000 1 1012 1004 5

B) 1000 1000 1000 1012 1004 1004

C) 1000 1000 1 1012 1008 5

☒ D) None

~~**a~~ $\Rightarrow$ ~~**a[0]~~

$\Rightarrow$ *a[0]

~~*a[0][0]~~

$\Rightarrow$ a[0][0]

2

Q Gate-2005

$\text{int} (*f)(\text{int}^*);$

- ~~A)~~ A function that takes an int. pointer as arg. & returns an integer.
- ~~B)~~ A func. that takes an int. as arg. & returns a int. pointer.
- C) A pointer to a function that takes an int. pointer as arg. and returns an integer.
- ~~D)~~ A function that takes an integer pointer as arg. & returns a func. pointer.

Pointer₁ - Pointer₂

```
int *P1 ;  
int *P2 ;  
P1 = &a[3] ;  
P2 = &a[0] ;
```

a[0]	a[1]	a[2]	a[3]	a[4]
10	20	30	40	50

1000	1004	1008	1012	1016
------	------	------	------	------

$P_1 - P_2$

P_2

P_1

$$\frac{\text{Actual diff}}{(4)} = \frac{1012 - 1000}{4} = \frac{12}{4} = (3)$$

Q

int a[4] = {10, 20, 30, 40};

int *p[4] = {a+3, a+2, a+1, a};

int y;

y = --P[0] - P[1];

printf("/d", y);

printf("/d", *p[0]);

a[0]	a[1]	a[2]	a[3]
10	20	30	40
100	104	108	112

$$(i) P[0] = P[0] - 1$$

$$= \&a[3] - 1$$

$$= \&a[2]$$

$$(ii) y = P[0] - P[1]$$

$$= \frac{108 - 108}{4}$$

$$y = 0$$

&a[3]	&a[2]	&a[1]	&a[0]
P[0]	P[1]	P[2]	P[3]

A) 0 20

B) 1 30

C) 0 30

D) 0 10

$$*P[0] \Rightarrow \&a[2] = a[2] \Rightarrow \underline{\underline{30}}$$

Q

```
int a = 5, b = 10, c = 15;
```

```
int *p[3] = {&a, &b, &c};
```

```
printf("/d", *p[*p[1]-8]);
```

NULL pointer

A) Segmentation fault

B) Compilation Error

C) 15

D) 10

$\Rightarrow$ ~~*&c~~
 $\Rightarrow$ (C)

(15)

$\rightarrow$ ~~*p~~ [*&b - 8]

$\rightarrow$ p[b - 8]

$\rightarrow$ p[10 - 2] $\Rightarrow$ *p[2]

Q Gate 2015

```
static int a[] = { 10, 20, 30, 40, 50};
```

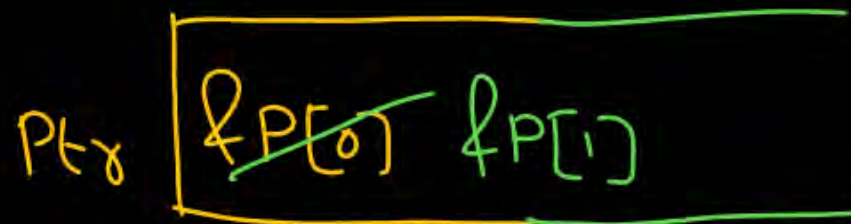
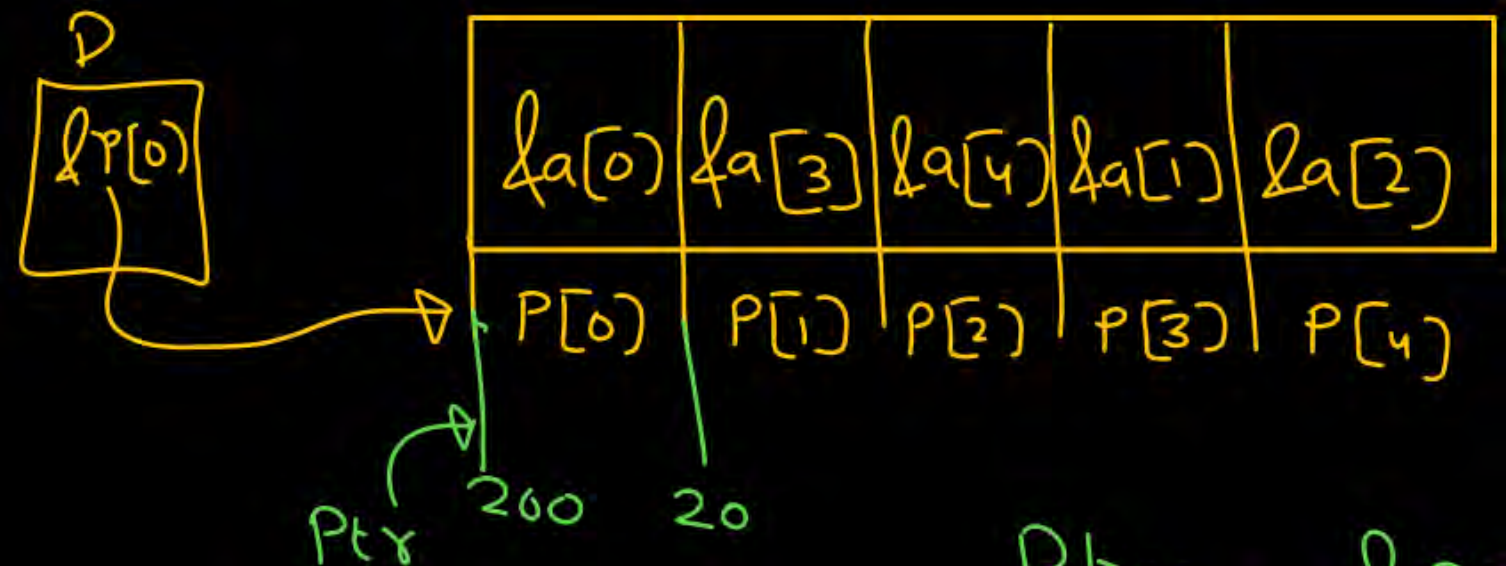
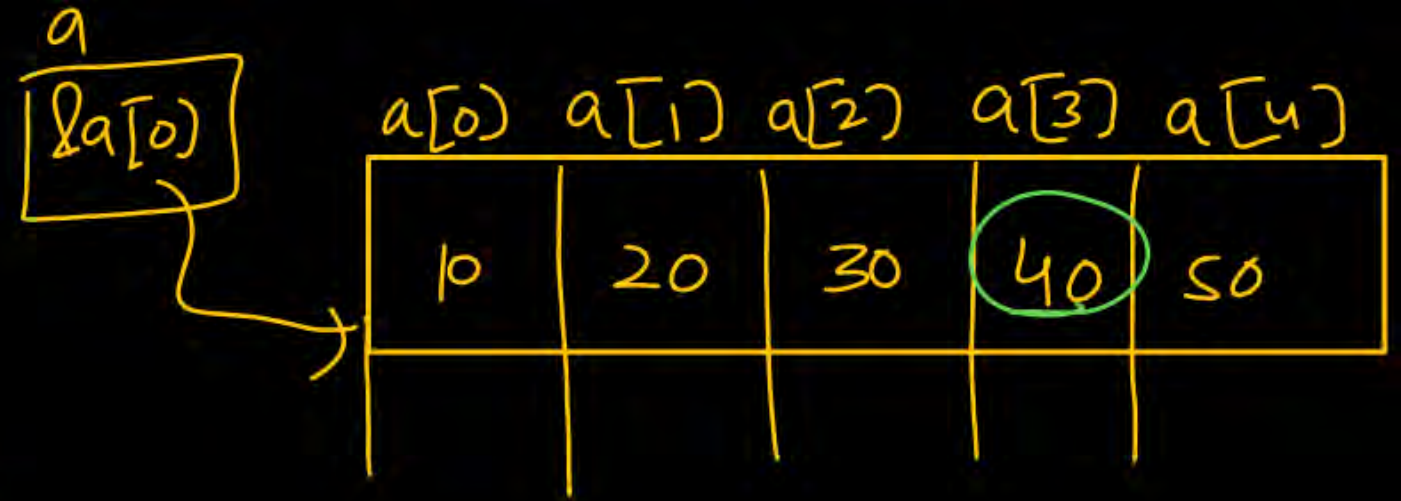
```
static int *p[] = { a, a+3, a+4, a+1, a+2};
```

```
int **ptr = p;
```

```
ptr++;
```

```
printf("%d %d", ptr-p, **ptr);
```

$\&P[1] - \&P[0]$
 $\Rightarrow \frac{204 - 200}{4} = 1$



$ptr = \&p[1]$
 $*ptr = * \&p[1]$
 $*ptr = p[1]$
 $*ptr = \&a[3]$

$**ptr = * \&a[3] = a[3]$


```
int f1(){  
    printf("Pankaj");  
    return 1;  
}
```

```
int f2(){  
    printf("Sharma");  
    return 0;  
}
```

```
int main(){
```

```
    int (*P[2])();
```

```
    P[0] = f1; ✓
```

```
    P[1] = f2; ✓
```

P[0]();

return 0;

}

f1();

Calling

Pankaj

→ P is an array of 2 pointer to function that takes no arg & returns an int.

Gate 2003

```
#include <stdio.h>
```

```
#define print(x) printf("/d", x);
```

```
int x;
```

```
void Q(int z) {  
    z = z + x;  
    print(z);  
}
```

```
void P(int* y) { int x = *y + 2;  
    Q(x);  
    *y = x - 1;  
    print(x);  
}
```

```
void main() {  
    x = 5;  
    P(&x);  
    print(x);  
}
```

A 12 7 6 B 22 12 11
C 14 6 6 D 7 6 6

Gate-2003

int *A[10];

int B[10][10];

of the following exprs →

1) A[2]

2) A[2][3]

~~3) B[1]~~

4) B[2][3]

* (A[2] + 3)

Add

A 1, 2 and 4

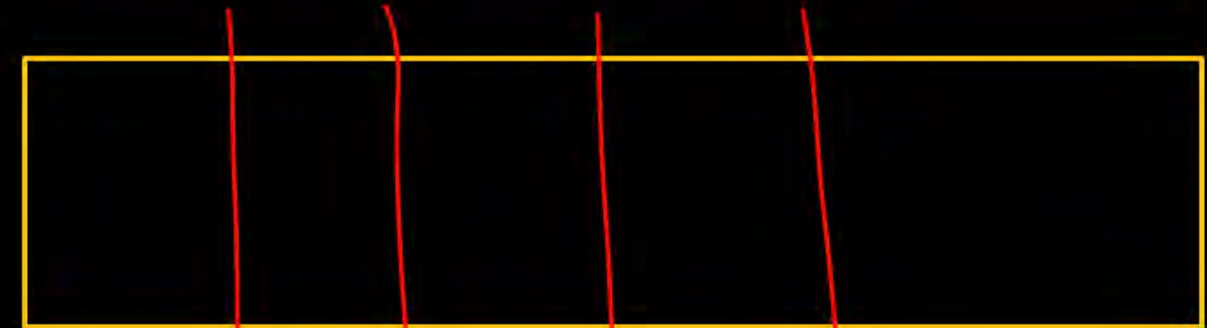
B 2, 3 and 4

C 2 and 4

D 4 only

can add
can take int
Which will not
give C.T.E. if

used as Lvalue of
an assignment stat.



A[0] | A[1] | A[2] | A[3] | ...

A[0] = 0

Gate 2020

int a[4][5] = { {1,2,3,4,5},
 {6,7,8,9,10},
 {11,12,13,14,15},
 {16,17,18,19,20}};

printf("%d\n", *(*(a + **a + 2) + 3));

19

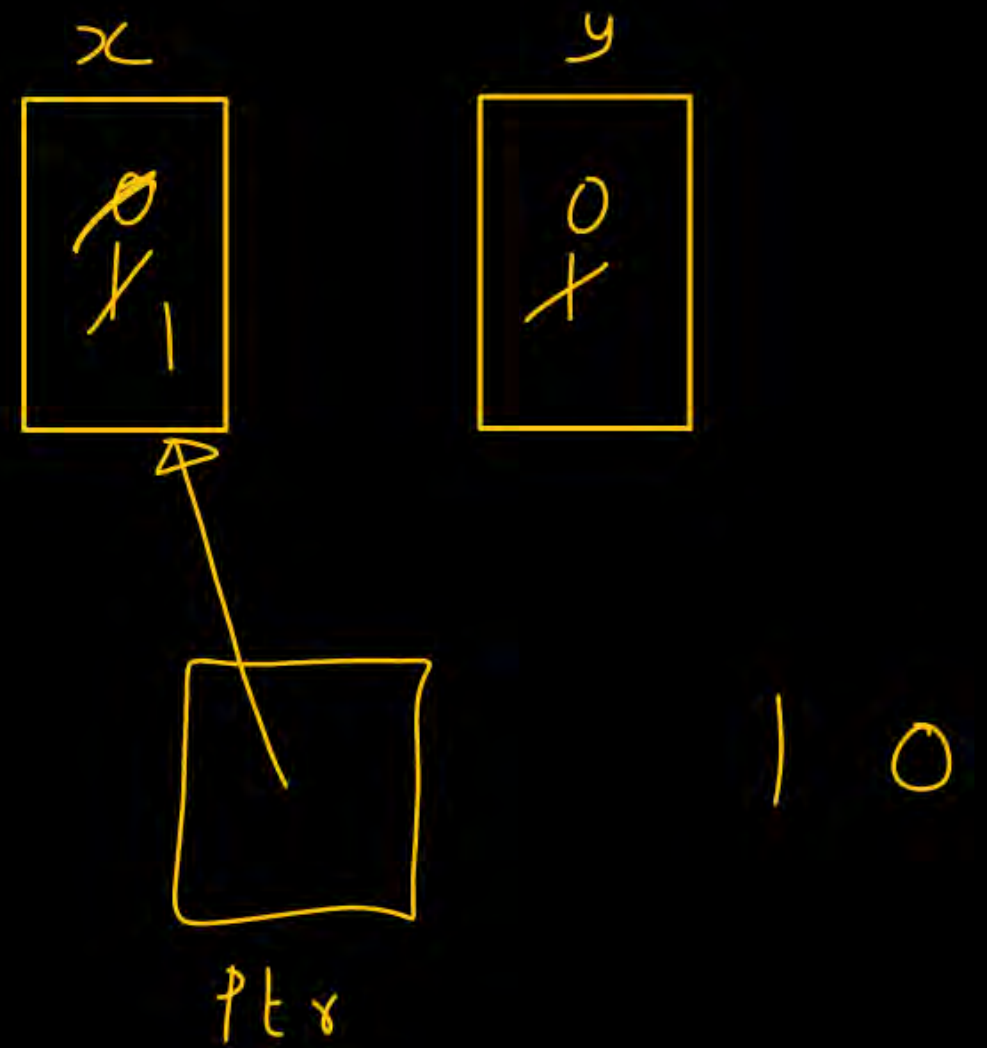
$$*(a + **a + 2 + 3)$$

$$= *(a[3] + 3)$$

$$= a[3][3]$$

Gate-2017

```
void printxy(int x, int y){  
    int *ptx;  
    x = 0;  
    ptx = &x;  
    y = *ptx;  
    *ptx = 1;  
    printf("/.d /.d", x, y);  
}
```



The o/p of invoking $\text{printxy}(1,1)$

A) 0 0

B) 0 1

C) 1 0

D) 1 1

```
main(){  
    printxy(1,1);  
}
```

Gate 2003

12 7 6

void main() {

#include <stdio.h>

#define print(x) printf("/d", x);

int x;

void Q(int z) {
 z = z + x;
 print(z);
}

void P(int* y) { int x = *y + 2;
 Q(x);
 *y = x - 1;
 print(x);
}

x = 5;

P(&x);

print(x);

}

y
P(1016)

